

InterviewForge AI

Task Division & Execution Tracker — 8 Weeks, 3-Member Team

This is a live tracker, not a plan document. Update the Status column weekly as work progresses.

Team & Domain Ownership

Member	Primary Domain
Krishna (Frontend)	Frontend — all UI screens, client-side logic, responsive/accessible layout
Akhilesh (Backend & DB)	Backend & Database — API, auth, Prisma schema, deployment/DevOps
Ayush (AI Integration)	AI Integration — Gemini prompts, resume parsing, evaluation & report generation logic

Weekly Task Tracker

Week	Task	Owner	Status
1	Wireframe core screens (login, dashboard, resume upload, interview, report, history)	Krishna (Frontend)	To Do
	Design DB schema / ERD (Users, Resumes, Interviews, Questions, Responses, Reports)	Akhilesh (Backend & DB)	To Do
	Set up GitHub repo, branching strategy, project board	All 3	To Do
	Create free-tier accounts: Vercel, Render, Supabase	Akhilesh (Backend & DB)	To Do
	Create Gemini API key (Google AI Studio)	Ayush (AI Integration)	To Do
2	Express project structure + Prisma migrations	Akhilesh (Backend & DB)	To Do
	Register/login/refresh/logout endpoints (JWT)	Akhilesh (Backend & DB)	To Do
	Role-based access middleware	Akhilesh (Backend & DB)	To Do
	Login & Signup screens (Next.js + Tailwind)	Krishna (Frontend)	To Do
	Wire Login/Signup UI to real auth API	Krishna (Frontend)	To Do
	Deploy staging backend (Render) + staging frontend (Vercel)	Akhilesh (Backend & DB)	To Do
3	Interviews / Questions / Responses tables & endpoints	Akhilesh (Backend & DB)	To Do
	Interview Setup screen (category → role → mode → start)	Krishna (Frontend)	To Do
	Live Interview screen (question card, timer, text input, progress bar)	Krishna (Frontend)	To Do
	Placeholder-question session flow end-to-end	Krishna (Frontend)	To Do
	Define API contract for question/response payloads (shared with AI module)	Akhilesh (Backend & DB)	To Do
4	Build isolated AI Service module (generateQuestions, evaluateResponse, generateReport)	Ayush (AI Integration)	To Do

	Gemini integration – Technical + HR question generation	Ayush (AI Integration)	To Do
	JSON-schema validation of AI responses before backend trusts them	Ayush (AI Integration)	To Do
	Replace placeholder questions in Live Interview screen with real AI output	Krishna (Frontend)	To Do
	Backend endpoint to call AI Service and store results	Akhilesh (Backend & DB)	To Do
5	Resume upload to Supabase Storage (PDF/DOCX)	Akhilesh (Backend & DB)	To Do
	AI resume-parsing prompt → structured JSON output	Ayush (AI Integration)	To Do
	Resume review/edit screen for the student	Krishna (Frontend)	To Do
	Wire parsed resume data into question-generation prompt	Ayush (AI Integration)	To Do
6	Report generation logic on session completion	Ayush (AI Integration)	To Do
	Results/Report screen with question-by-question feedback	Krishna (Frontend)	To Do
	History + score-trend view (Recharts)	Krishna (Frontend)	To Do
	Backend endpoints for report/history data	Akhilesh (Backend & DB)	To Do
	Stretch: Web Speech API integration for voice input	Krishna (Frontend)	To Do
7	Structured test pass: empty resume, AI timeout, abandoned session, slow network	All 3	To Do
	Cross-browser + mobile breakpoint check	Krishna (Frontend)	To Do
	Fix defects found in testing	All 3	To Do
	UI/spacing/loading-state polish	Krishna (Frontend)	To Do
	Finish/stabilize voice mode or confirm text-mode fallback	Ayush (AI Integration)	To Do
8	Finalize production deployment (Vercel + Render + Supabase)	Akhilesh (Backend & DB)	To Do
	Set production environment variables/secrets	Akhilesh (Backend & DB)	To Do
	Write final README + API notes	Akhilesh (Backend & DB)	To Do
	Write individual contribution summary	All 3	To Do
	Prepare and rehearse demo script	All 3	To Do
	Record fallback demo video	Krishna (Frontend)	To Do

Shared / Dependent Task Review

Some tasks touch more than one member's domain. These require a review from all 3 before merging into main:

- API contracts (request/response shape between Frontend, Backend, and AI Service) – changed only after a quick 3-way check so no one builds against a stale shape
- Database schema changes – Backend proposes, all 3 confirm before running a migration, since Frontend and AI modules both depend on field names/types
- AI response JSON schema – AI Integration proposes, Backend confirms it can validate/

store it, Frontend confirms it can render it

- Any change to an already-merged shared file (e.g., shared types/interfaces) – flagged in the team channel before pushing

Rule of thumb: if a change only affects your own domain folder, merge directly. If it changes something another member's code calls or depends on, get a quick sign-off first – a short message is enough, no formal review process needed for a 3-person team.